UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

CS2024 - Theory of Computation

First upgrade

Angel Napa

TRANSFORMATEC

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

1.

Recap

Previously, on Computation Theory

TRANSFORMATEC

Theorem

REGULAR is closed under concatenations.

If L_1 and L_2 are two regular languages, then $L_1 \cdot L_2$ is also a regular language

Proof Idea

- Spooky
- How can a FDA know the right partition to the string such that the first half is accepted by A1 and the second by A2?
- We need a more permissive model
- We need **NONDETERMINISM**

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

2.

NonDeterministic

Finite Automaton

TRANSFORMATEC

https://www.youtube.com/watch?v=lufECeWtN34

UTEC UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

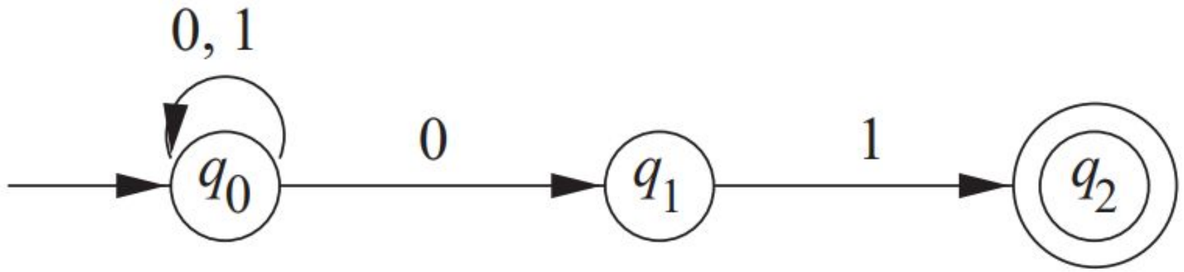
> Reinventa el mundo <

NonDeterminism

- **Determinism**
  - Given a state, and a symbol, **we know exactly** what state we should go.
- **Nondeterminism**
  - Given a state, and a symbol, **There are several** (possibly empty) choices.
  - Nicolas Cage being a NFA
  - To understand smoothly this, we will separate the NFA in two: NFA and **NFA-ε**
    - This is Hopcroft Approach, Sipser doesn't differentiate these 2 kinds of NFA(because he is a pro)

TRANSFORMATEC

A (NonDeterministic Finite) *automaton* M_1



- **State Diagram:** Figure
- **States:** q_0, q_1, q_2
- **Start state:** q_0
- **Set of accept states:** $\{q_2\}$
- **Transitions:** the arrows

NFA (friendly NFA)

DEFINITION 1.37

A ***nondeterministic finite automaton*** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta : Q \times \Sigma \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Formal Definitions

Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA and w a string over the alphabet Σ. Then we say that N ***accepts*** w if we can write w as $w = y_1 y_2 \cdots y_m$, where each y_i is a member of Σ and a sequence of states $r_0, r_1, \ldots, r_m$ exists in Q with three conditions:

1. $r_0 = q_0$,
2. $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i = 0, \ldots, m-1$, and
3. $r_m \in F$.

NFA-ε (just NFA for Sipser)

DEFINITION 1.37

A ***nondeterministic finite automaton*** is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$, where

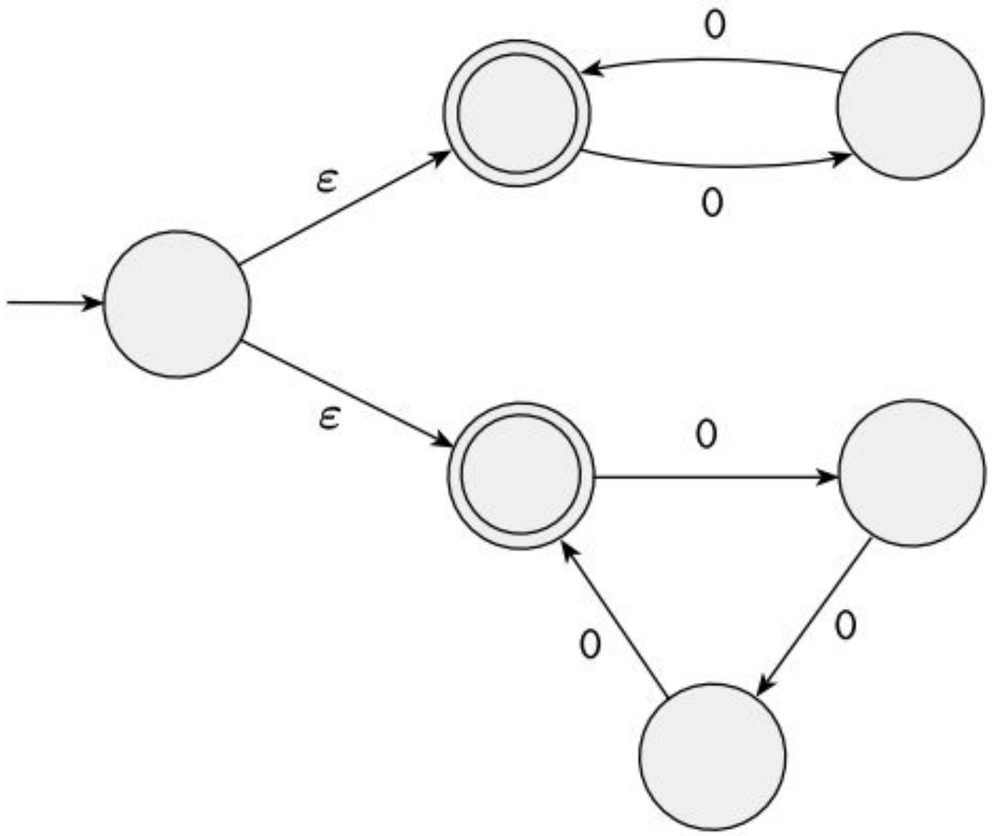
1. Q is a finite set of states,
2. Σ is a finite alphabet,
3. $\delta\colon Q \times \Sigma_\varepsilon \longrightarrow \mathcal{P}(Q)$ is the transition function,
4. $q_0 \in Q$ is the start state, and
5. $F \subseteq Q$ is the set of accept states.

Formal Definitions

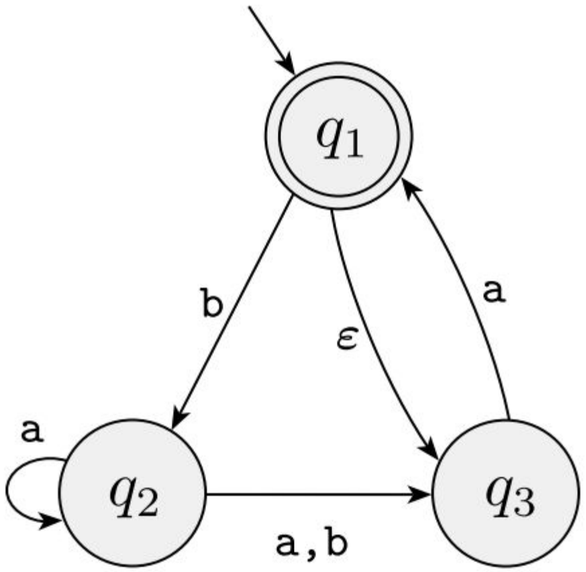
Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA and w a string over the alphabet Σ. Then we say that N ***accepts*** w if we can write w as $w = y_1 y_2 \cdots y_m$, where each y_i is a member of Σ_ε and a sequence of states $r_0, r_1, \ldots, r_m$ exists in Q with three conditions:

1. $r_0 = q_0$,
2. $r_{i+1} \in \delta(r_i, y_{i+1})$, for $i = 0, \ldots, m-1$, and
3. $r_m \in F$.

- What strings does this NFA recognizes?



A real *NFA-ε*



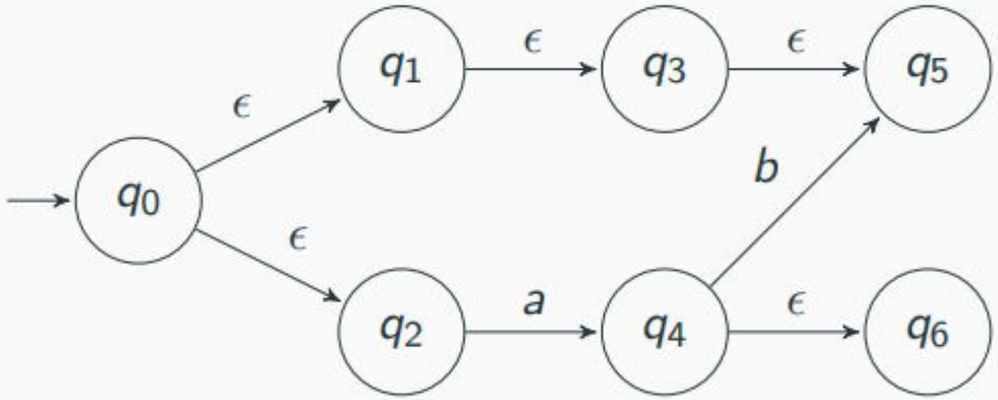
- **Is babaaa recognized by this NFA?**
- Quite confusing if we only use the previous formal definition
- Many ways to decompose a string with addition of **ε**: babaaa, b**ε**a**ε**baaa, **εεε**babaaa, etc..
- This notation will help us a little
- **ε-Closure of a state.**

ε-**Closure of** *a state, and a Set*

- For a state q, let **E(q)** denote the set of states that can be reach from q, by following zero or more **ε**-transitions.
- **E(q)** is is known as the **epsilon closure of q**
- The **ε-closure of a set of states** S, is defined as the union of the **ε**-closures of each of the states that are part of S. This is
  - **E(S)** = $\cup_{q \in S} E(q)$

ε-Closure of *a state, and a Set*

Indique cuales son las clausuras de cada uno de los estados del siguiente autómata



Alternative Definition (NFA-ε)

The formal definition of computation for an NFA is similar to that for a DFA. Let $N = (Q, \Sigma, \delta, q_0, F)$ be an NFA and w a string over the alphabet Σ. Then we say that N ***accepts*** w if we can write w as $w = y_1 y_2 \cdots y_m$, where each y_i is a member of Σ and a sequence of states $r_0, r_1, \ldots, r_m$ exists in Q with three conditions:

- $r_0 \in E(q_0)$
- $r_{i+1} \in E(\delta(r_i, y_{i+1}))$, for i = 0, . . . , m-1, and
- $r_m \in F$

Definition

- N **recognizes** a language L if

  L = {ω| N accepts ω}

- We also write **L = L(N)**

Class Activity

Diseñe AFNs para los siguientes lenguajes

a. Cadenas, con alfabeto {0, 1}, que tienen a 0101 como subcadena
b. Cadenas, con alfabeto {0, 1, 2, 3}, cuyo último dígito ya apareció antes

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

3.

NFA vs DFA

Is NonDeterminism here more powerful?

TRANSFORMATEC

Main Question

- If A is DFA, then A is also NFA
- If L is regular, then is also recognizable by some NFA
- By definition, it could be some language recognizable by a NFA might not be regular
- **Do NFA and DFA recognize the same class of languages?**

Main Question

- If A is DFA, then A is also NFA
- If L is regular, then is also recognizable by some NFA
- **Def:** Two Machines are equivalent if they recognize the same language
- By definition, it could be some language recognizable by a NFA might not be regular
- **Do NFA and DFA recognize the same class of languages?**

Theorem

THEOREM 1.39

Every nondeterministic finite automaton has an equivalent deterministic finite automaton.

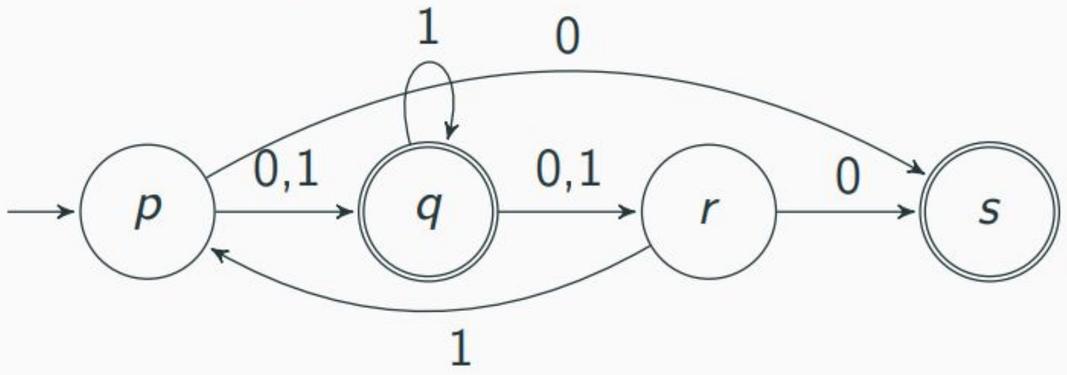
Proof Idea (NFA)

- Proof by construction
- Let N be a NFA.
- Let L = L(N)
- We will build explicitly an DFA M, such that L = L(M)
  - Each transition of M should simulate the many options of N.
  - This can be done if we know all these options
  - Save options? That's just a set
  - First we will see easy version

Proof Idea

- Let $N = \{Q, \Sigma, \delta, q_0, F\}$
- Let $L = L(N)$
- We define $A = \{Q', \Sigma, \delta', q'_0, F'\}$ as
- $Q' = \mathcal{P}(Q)$
- $\delta'(S, a) = \cup_{q \in S} \delta(q, a)$
- $q'_0 = \{q_0\}$
- $F' = \{S \in Q'$: S has an element of $F\}$

(a) Convertir el siguiente AFN en AFD:



(b) Diseñe un AFN para el siguiente lenguaje: cadenas, con alfabeto $\{0, 1\}$, que tienen a 0101 como subcadena. Luego conviértalo a un AFD con la técnica vista.

Proof Idea

- Let $N = \{Q, \Sigma, \delta, q_0, F\}$
- Let $L = L(N)$
- We define $A = \{Q', \Sigma, \delta', q'_0, F'\}$ as
- $Q' = \mathcal{P}(Q)$
- $\delta'(S, a) = \cup_{q \in S} E(\delta(q, a)) = E(\cup_{q \in S} \delta(q, a))$
- $q'_0 = E(q_0)$
- $F' = \{S \in Q'$: S has an element of $F\}$

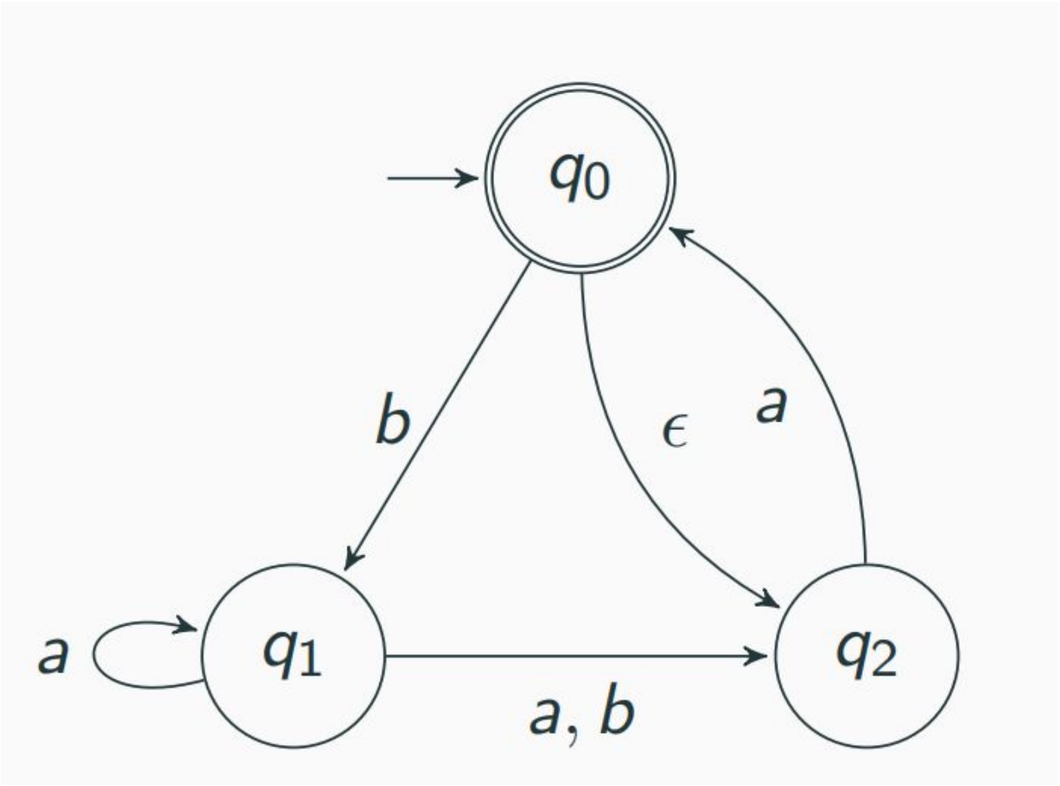
Conversion NFA to DFA

- **Step 0:** $q'_0 = E(q_0)$, $P_1 = \{q'_0\}$, $i = 1$
- **Step i:** For each symbol a, and each state $p \in \delta'(p,a)$
- Let P_{i+1} the new set of states created in step i
- If $P_{i+1} = \varnothing$
  - Finish algorithm
- Else
  - Execute **step i+1**

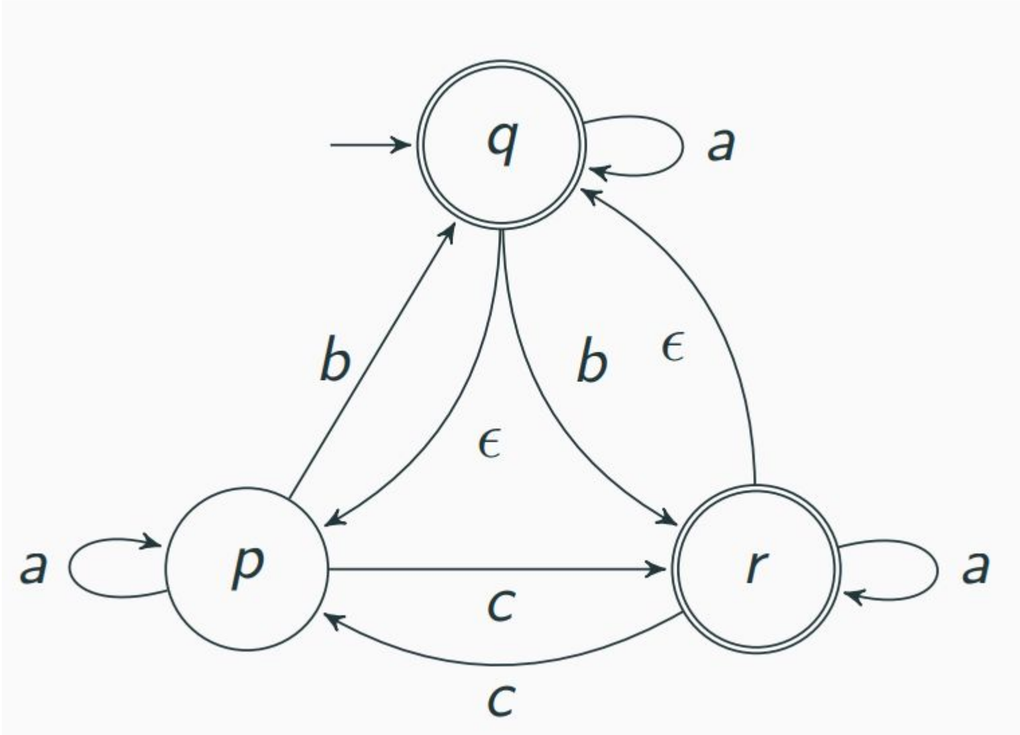
Class Activity

- For each of the following NFA
  - Find the closure of each state, and each set of states
  - Compute the chain babb
  - Build an equivalent DFA

Class Activity



Class Activity



UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

4.

Regular Operations

Are they closed by them?

TRANSFORMATEC

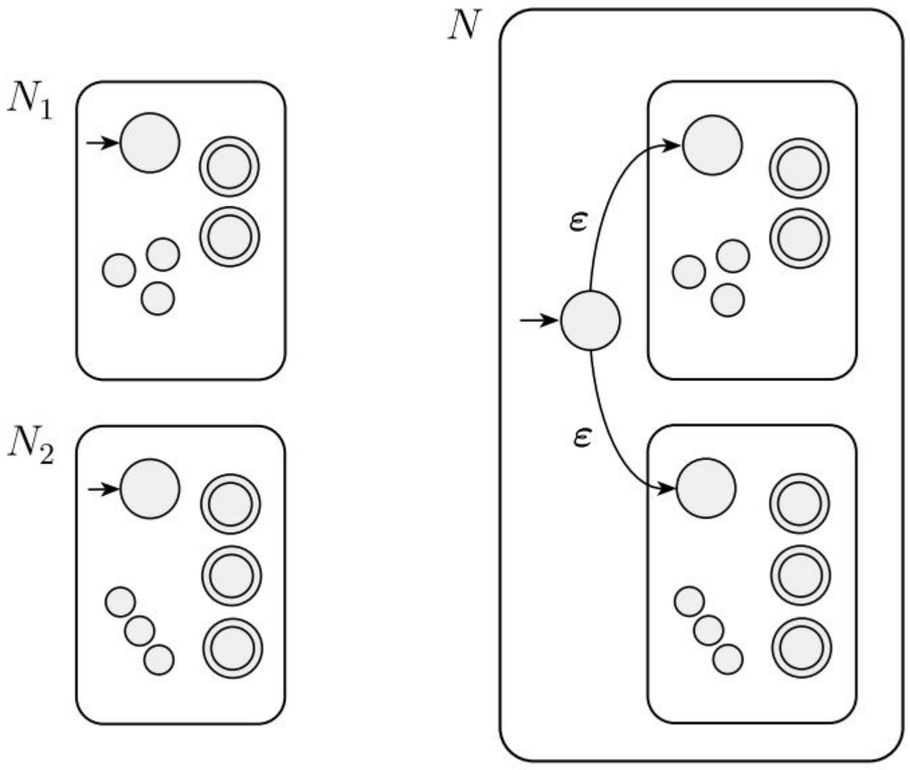
Theorem

REGULAR is closed under union

Proof Idea

- Let L_1 and L_2 regular languages
- Let N_1 and N_2 NFAs that recognizes L_1 and L_2
- We create N, such that N accepts a string if either of N_1 or N_2 accepts this input.
- We will create this possibility with the ε arrows

Proof Idea



Proof Idea

- Let $L = L_1 \cup L_2$
- Let $N_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$
- Let $N_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$
- We define $N = \{Q, \Sigma, \delta, q_0, F\}$ as
- $Q = \{q_0\} \cup Q_1 \cup Q_2$
- $F = F_1 \cup F_2$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \\ \delta_2(q,a) & q \in Q_2 \\ \{q_1, q_2\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon. \end{cases}$$

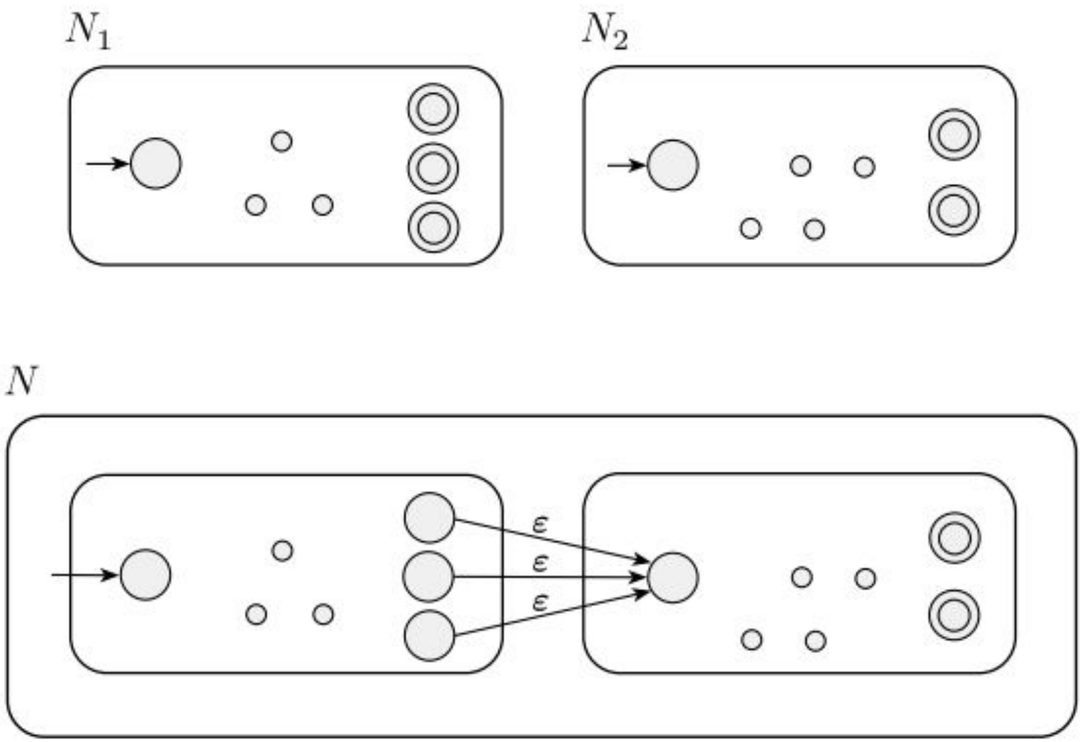
Theorem

REGULAR is closed under concatenation

Proof Idea

- Let L_1 and L_2 regular languages
- Let N_1 and N_2 NFAs that recognizes L_1 and L_2
- We create N, such that N accepts a string if first it we reach and accepted state in N_1 and then, go to an accepted state in N_2

Proof Idea



Proof Idea

- Let $L = L_1 . L_2$
- Let $N_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$
- Let $N_2 = \{Q_2, \Sigma, \delta_2, q_2, F_2\}$
- We define $N = \{Q, \Sigma, \delta, q_1, F\}$ as
- $Q = Q_1 \cup Q_2$
- $F = F_2$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ and } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_2\} & q \in F_1 \text{ and } a = \varepsilon \\ \delta_2(q,a) & q \in Q_2. \end{cases}$$

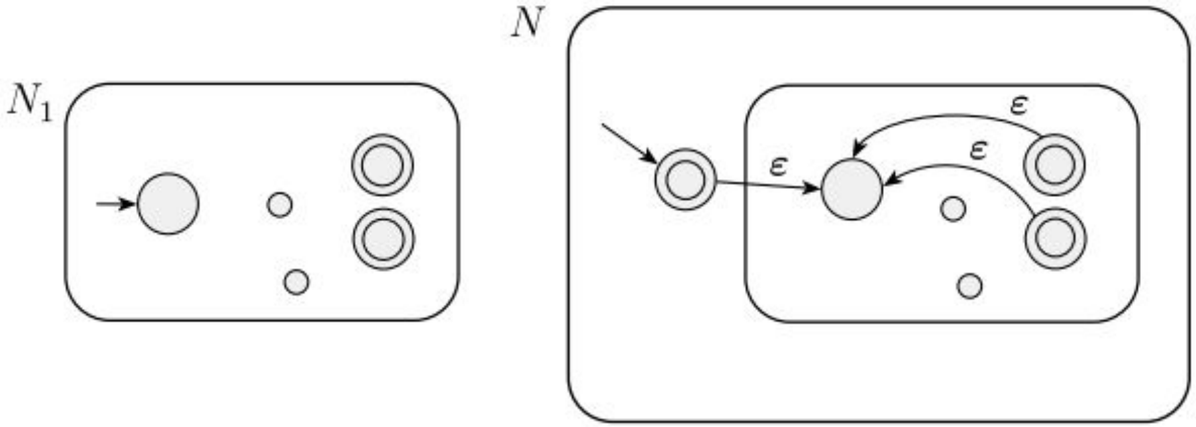
Theorem

REGULAR is closed under star operation

Proof Idea

- Let L_1 and L_2 regular languages
- Let N_1 and N_2 NFAs that recognizes L_1 and L_2
- We create N, such that N accepts a string if first it we reach and accepted state in N_1 and then, go to an accepted state in N_2

Proof Idea



Proof Idea

- Let $L = L_1^*$
- Let $N_1 = \{Q_1, \Sigma, \delta_1, q_1, F_1\}$
- We define $N = \{Q, \Sigma, \delta, q_0, F\}$ as
- $Q = Q_1 \cup \{q_0\}$
- $F = \{q_0\} \cup F_1$

$$\delta(q,a) = \begin{cases} \delta_1(q,a) & q \in Q_1 \text{ and } q \notin F_1 \\ \delta_1(q,a) & q \in F_1 \text{ and } a \neq \varepsilon \\ \delta_1(q,a) \cup \{q_1\} & q \in F_1 \text{ and } a = \varepsilon \\ \{q_1\} & q = q_0 \text{ and } a = \varepsilon \\ \emptyset & q = q_0 \text{ and } a \neq \varepsilon. \end{cases}$$

Exercises

- Let $L_1 = \{w: |w| < 6\}$
- Let $L_2 = \{w: \text{ends in } 01\}$
- Build AFN-ε for the following languages
  - $L_1 \cup L_2$
  - $L_1 . L_2$
  - L_1^*
  - L_2^*

UTEC
UNIVERSIDAD DE INGENIERÍA Y TECNOLOGÍA

> Reinventa el mundo <

Thanks

Let’s keep finding mistakes

TRANSFORMATEC